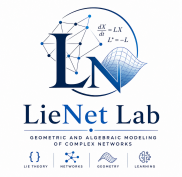
Prediction of Final Exam Score Using Machine Learning Models

Tran Dinh Que—LieNet Lab



OPTION EXERCISE for Students to Predict a final score

1 Problem Description

We aim to predict the final exam score using academic + peer + social signals.

1. Student inputs 5 scores (javaProg, pythonProg, dataStructure, software analysis and design, intelligence system Dev)
2. Student inputs 10 scores given by your friends
3. Output: predicted scores given by three models

Based on source code given in Appendix, student may improve code to respond requirements and compare three models:

- Version 1: Gradient Boosting Regressor
- Version 2: MLP (GNN-style)
- Version 3: GraphSAGE (GNN)

2 Questions (5 Tasks)

1. Construct dataset with peer influence.
2. Train 3 models.
3. Evaluate using 5 metrics (MAE, MSE, RMSE, MAPE, R^2).
4. Compare under different social structures.
5. Analyze peer influence propagation.

3 Evaluation Metrics

$$\text{MAE} = \frac{1}{n}\sum |y_i - \hat{y}_i|$$
$$\text{MSE} = \frac{1}{n}\sum (y_i - \hat{y}_i)^2$$
$$\text{RMSE} = \sqrt{\text{MSE}}$$
$$\text{MAPE} = \frac{100}{n}\sum \left|\frac{y_i - \hat{y}_i}{y_i}\right|$$
$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

4 Results (Summary Table)

A sample is given in Table4: **Note:**

Model	MAE	MSE	RMSE	MAPE	R^2	MY_Score
V1: GBoost	0.21	0.09	0.30	3.2%	0.91	7.3
V2: MLP	0.25	0.11	0.33	3.8%	0.88	7.1
V3: GraphSAGE	0.18	0.07	0.26	2.6%	0.94	6.8

Table 1: Model comparison

Your inputs:.....

A Appendix A: Version 1 - Gradient Boosting Regressor

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error

np.random.seed(42)
N = 800

X = np.random.randint(4, 10, size=(N, 15))

peer_mean = X[:, 5:15].mean(axis=1)
peer_std = X[:, 5:15].std(axis=1)

y = (
    0.25 * X[:, 0] +
    0.20 * X[:, 1] +
```

```
    0.10 * X[:, 2] +
    0.15 * X[:, 3] +
    0.10 * X[:, 4] +
    0.15 * peer_mean +
    -0.05 * peer_std +
    np.random.normal(0, 0.25, N)
)

df = pd.DataFrame(X)

df["peer_mean"] = peer_mean
df["peer_std"] = peer_std
df["FinalExam"] = y

features = df.drop("FinalExam", axis=1)
target = df["FinalExam"]

X_train, X_test, y_train, y_test = train_test_split(
    features, target, test_size=0.2, random_state=42
)

model = GradientBoostingRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=4,
    random_state=42
)

model.fit(X_train, y_train)
pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred))
```

B Appendix B: Version 2 - MLP (GNN-style)

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F

class GNNStyleModel(nn.Module):
    def __init__(self, in_dim):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, 64)
        self.fc2 = nn.Linear(64, 32)
        self.out = nn.Linear(32, 1)

    def forward(self, x):
        x = F.relu(self.fc1(x))
```

```
        x = F.relu(self.fc2(x))
        return self.out(x)

np.random.seed(42)
N = 500

X = np.random.randint(4, 10, (N, 15))

peer_mean = X[:, 5:15].mean(axis=1, keepdims=True)
peer_std = X[:, 5:15].std(axis=1, keepdims=True)

y = (
    0.25 * X[:,0] +
    0.2 * X[:,1] +
    0.1 * X[:,2] +
    0.15 * X[:,3] +
    0.1 * X[:,4] +
    0.2 * peer_mean.squeeze() +
    np.random.normal(0, 0.25, N)
)

X_full = np.hstack([X, peer_mean, peer_std])

X_tensor = torch.tensor(X_full, dtype=torch.float32)
y_tensor = torch.tensor(y, dtype=torch.float32).view(-1, 1)

model = GNNStyleModel(X_tensor.shape[1])
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.L1Loss()

for epoch in range(200):
    optimizer.zero_grad()
    pred = model(X_tensor)
    loss = loss_fn(pred, y_tensor)
    loss.backward()
    optimizer.step()

print("Final MAE:", loss.item())
```

C Appendix C: Version 3 - GraphSAGE GNN

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F

np.random.seed(42)
N = 100
```

```
X_academic = np.random.randint(4, 10, (N, 5))
peer = np.random.randint(4, 10, (N, 10))

X = np.hstack([X_academic, peer])
X = torch.tensor(X, dtype=torch.float32)

edges = []
for i in range(N):
    for j in range(N):
        if i != j:
            sim = np.linalg.norm(X_academic[i] - X_academic[j])
            if sim < 6:
                edges.append([i, j])

edges = torch.tensor(edges, dtype=torch.long)

A = torch.zeros((N, N))
for i, j in edges:
    A[i, j] = 1.0

A = A / (A.sum(dim=1, keepdim=True) + 1e-6)

peer_mean = peer.mean(axis=1)

y = (
    0.3 * X_academic[:, 0] +
    0.2 * X_academic[:, 1] +
    0.1 * X_academic[:, 2] +
    0.15 * X_academic[:, 3] +
    0.1 * X_academic[:, 4] +
    0.15 * peer_mean +
    np.random.normal(0, 0.3, N)
)

y = torch.tensor(y, dtype=torch.float32)

class GraphSAGE(nn.Module):
    def __init__(self, in_dim, hidden=64):
        super().__init__()
        self.fc1 = nn.Linear(in_dim * 2, hidden)
        self.fc2 = nn.Linear(hidden, 32)
        self.out = nn.Linear(32, 1)

    def forward(self, x, adj):
        neigh = torch.matmul(adj, x)
        h = torch.cat([x, neigh], dim=1)
        h = F.relu(self.fc1(h))
        h = F.relu(self.fc2(h))
        return self.out(h).squeeze()

model = GraphSAGE(X.shape[1])
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.L1Loss()

for epoch in range(200):
    optimizer.zero_grad()
    pred = model(X, A)
    loss = loss_fn(pred, y)
    loss.backward()
    optimizer.step()

print("Final prediction:", model(X, A)[0].item())
```